Week 5: Data Pipelines

Keeyong Han

Table Of Contents

1. Recap of the 4th Week
2. Quiz #1 Review
3. 2 Main Resume Mistakes
4. Overview of Data Pipelines
5. Key Concepts
6. Refreshing Python Syntax
7. First Data Pipeline (ETL)
8. Improving the Data Pipeline
9. More on Lab #1
10. Group Project & Final Exam
11. Resume Workshop (3/7)
12. Demo
13. Homework #4

Recap of the 4th Week

SQL

- The best language to manipulate structured data (small or big)
- The most important skill in the data domain
- Composed of two parts
  - DDL (Data Definition Language): CREATE/ALTER/DROP
  - DML (Data Manipulation Language): INSERT/DELETE/UPDATE/SELECT/UPSERT
    - COUNT DISTINCT
    - NULLIF, COALESCE
    - CTAS
    - Comparison against NULL

Quiz #1 Review

Quiz #1 Review

- Best 2 out of 3 quizzes will be graded
- The other two quizzes will follow the same format but slight changes
  - “Fill in the blank” questions will be added

2 Main Resume Mistakes

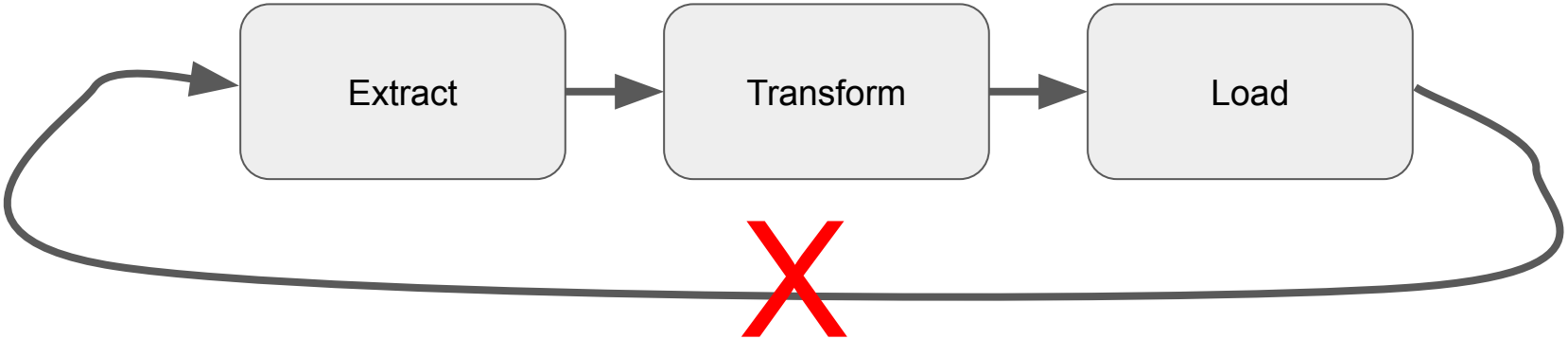
2 Mistakes I have seen over and over

- Missing one line header
- Too much irrelevant experience
  - Less is More

Overview of Data Pipelines

Definition of Data Pipeline

- A series of processes that automatically move and transform data from one or more sources to a destination
  - Mostly likely the destination is data warehouse or data lake
  - The source can be DW or DL itself in the case of ELT
  - Sometimes the flow is the other way around (Reverse ETL)
- In Airflow, data pipeline is called DAG (Directed Acyclic Graph)

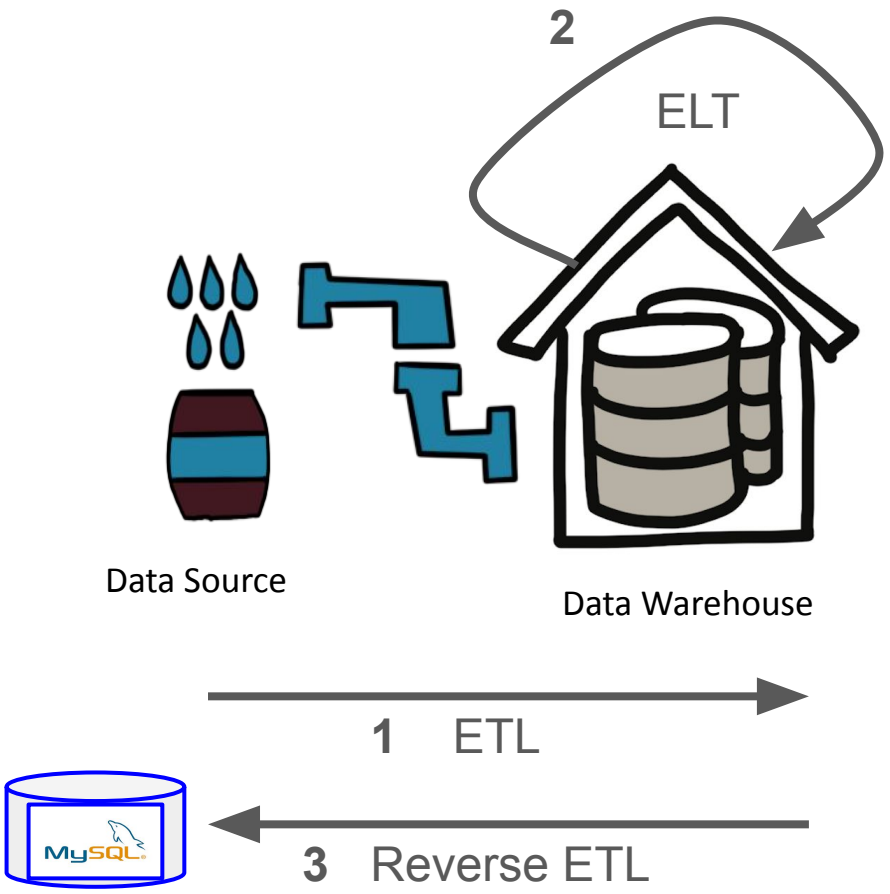


ETL (Extract, Transform, Load)

- Extract:
  - The process of reading data from the data source, usually through API calls.
- Transform:
  - The process of changing the format of the raw data into a desired form
- Load:
  - The final step of loading the data into the data warehouse as tables.

Types of Data Pipeline

1. ETL
2. ELT (“T”)
3. Reverse ETL



What is Airflow?

- An open source started from Airbnb along with Superset
- A de facto Data Pipeline management framework
  - You can write data pipelines and can manage them (scheduling, triggering, …)
  - Python modules are provided along with the Web UI
  - So popular that most Cloud providers supports Airflows as their service
    - Google Cloud supports Airflow (“Cloud Composer”)
- Data pipeline in Airflow is called DAG (Direct Acyclic Graph)

Apache Airflow

Key Concepts

Key Concepts to Look at

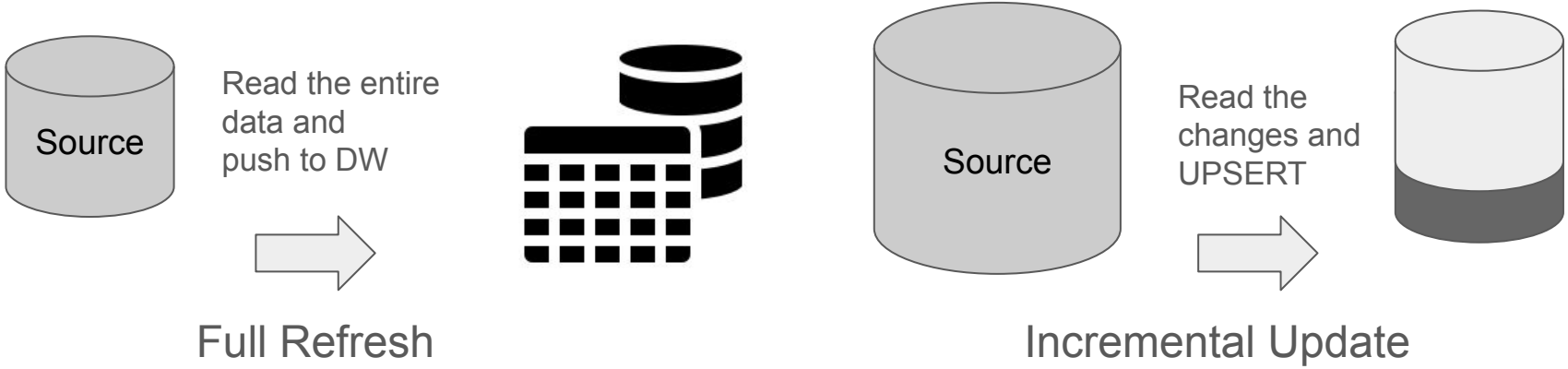
- Full Refresh, Incremental Update
- Idempotency
  - SQL Transaction along with Python
- Backfill
- Upsert
  - Most efficient way of ensuring the primary key uniqueness
  - Snowflake supports an operation called MERGE

Full Refresh vs. Incremental Update #1

- Full Refresh
  - Reading the whole data from the source and recreate the corresponding table
  - The simplest and cleanest approach but only works for small data source
- Increment Update
  - For large data sources, doing full refresh is not feasible
  - Doing periodic update such as daily or hourly
    - Daily incremental update: reading previous day’s data
    - Hourly incremental update: reading previous hour’s data
  - Effective but more complex to manage
    - What if the entire source data format changes?
    - What if updates from certain hours or days are found to be incorrect or missing?

Full Refresh vs. Incremental Update #2

- Prefer to do full-refresh if possible: everything gets easier
- If incremental update is the only way, the data source needs to support the followings:
  - Given a date/time range, returning records added or modified during the window.
  - Deletion shouldn't be allowed. If deletion is needed, introduce a deletion flag to mark.



Idempotency

- Running the data pipeline multiple times with the same input data should not change the contents of the final table
  - For example, duplicate data should not be created
- All critical points must be executed as one atomic action
  - This is why SQL transactions are an essential technique

Backfill

- Rerunning data pipelines to ensure complete & accurate data from the source
  - There are various reasons why past data might need to be re-read
- For full refresh, running the data pipeline again after fixing issues should work
  - Backfill isn’t really needed
- For incremental update, find missing or wrong time slots and rerun only those
  - If backfill is easy to perform, the lives of data engineers would be easier
    - Airflow makes the backfill easier (but learning it takes some efforts)
  - Ensuring the primary key uniqueness is required in some use cases

Upsert

- UPSERT = INSERT + UPDATE
  - Operates based on the Primary Key
  - If the record exists, update with new information
  - If the record doesn't exist, insert as a new record
  - What about deletion? Don’t delete records but flag a field (such as deleted)
- Data warehouses support UPSERT
  - Snowflake has a SQL operation called MERGE for this purpose

MERGE SQL: Snowflake’s Upsert

- Practice

Refreshing Python Syntax

What to learn: Basics

- Will use Google Colab

- Data Types
  - Number, string, list, dictionary, …
  - Slice operator
  - Formatted string literals
- For loop
- Functions
  - Definition and invocation

What to learn: More Advanced Topics

- Import modules
  - Installing modules via pip
- Downloading a web page: “request” module
  - This is in essence calling an API
- Error handling
  - try/except

Google Colab Practice

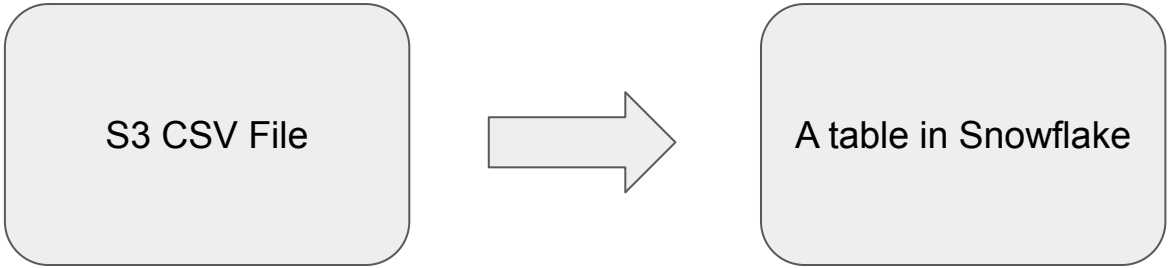
- Refreshing Python

Break

First Data Pipeline (ETL)

ETL Overview Practice

- Copy a country/capital CSV file from the web into a table in Snowflake
- Written in Python on Google Colab



ETL Overview - 1. Create a table in Snowflake

- Download the following file
  - https://s3-geospatial.s3.us-west-2.amazonaws.com/country_capital.csv
  - The file has two fields (country & capital)

country	capital
Abkhazia	Sukhumi
Afghanistan	Kabul
Albania	Tirana
Algeria	Algiers
American Samoa	Pago Pago

ETL Overview - 2. Create a table in Snowflake

```
CREATE TABLE raw.country_capital (
   country varchar  primary key,
   capital  varchar NOT NULL
);
```

ETL Overview - Overall Structure

- Written in Python on Colab: Consists of three functions (tasks)
  - extract, transform, load

```
def extract(url):
    # return the data at url
    …
    return data
```

→

```
def transform(data):
    # transform data into country,
    # capital list and return
    …
    return list
```

→

```
def load(list):
    # Insert records in list
    # to Snowflake
    …
    ...
```

ETL Practice

- [Data Pipeline Example from Google Colab]

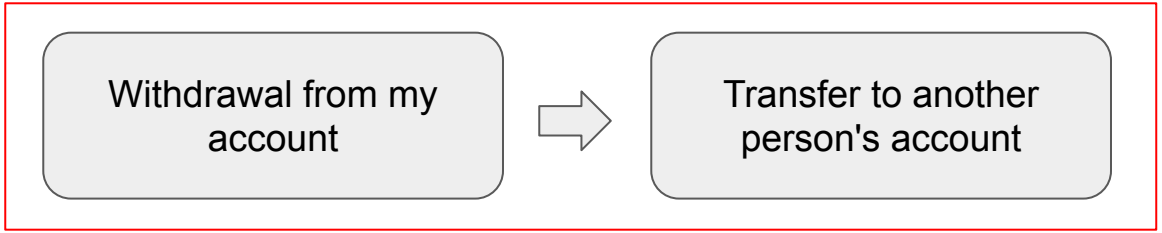
Improving the Data Pipeline

Three issues were identified

1. Header is currently being loaded as records.
2. Idempotency is not being maintained. Duplicate records are created each time the process is executed.
3. If the SQL fails midway, it could compromise data integrity.
    - Transactions should be used

What is SQL Transaction? (1)

- What if there are tasks that would be left in an incomplete state if they fail midway?
- In the example below, what if the withdrawal is successful but a problem occurs during the transfer?

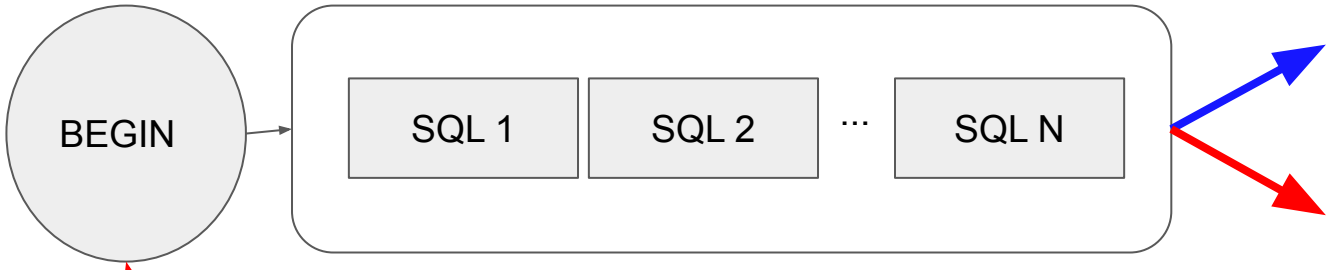


Bank transfer process: When I send money to another person

What is SQL Transaction? (2)

- Method for grouping SQLs that need to be executed as a single operation
- Use these SQL statements between BEGIN and END/COMMIT
- ROLLBACK (or ABORT) is an SQL that means "return to the state before BEGIN"
- In Snowflake, only DML are honored during transactions
  - DDL statements are committed immediately
    - MySQL is the same
  - SELECT doesn’t have to be protected. Data modification needs transactions

What is SQL Transaction? (3)



Transaction:

COMMIT/END: If all operations succeed, this temporary state becomes the final state.

ROLLBACK/ABORT: If even one operation fails midway, the system reverts to its original state

The SQL results during this process become a temporary state. They are not visible to other sessions until committed.

Best Practices in using SQL transaction with Python

- In Python, it is common to use try/except to handle errors
  - If an error occurs within the try/except block, a rollback is explicitly executed
  - If no error occurs, a commit is executed

```
try:
    cur.execute("BEGIN;")
    # execute SQLs that need to be run atomically
    …
    cur.execute("COMMIT;")  # same as cur.execute("END;")
except Exception as e:
    cur.execute("ROLLBACK;")
    raise
```

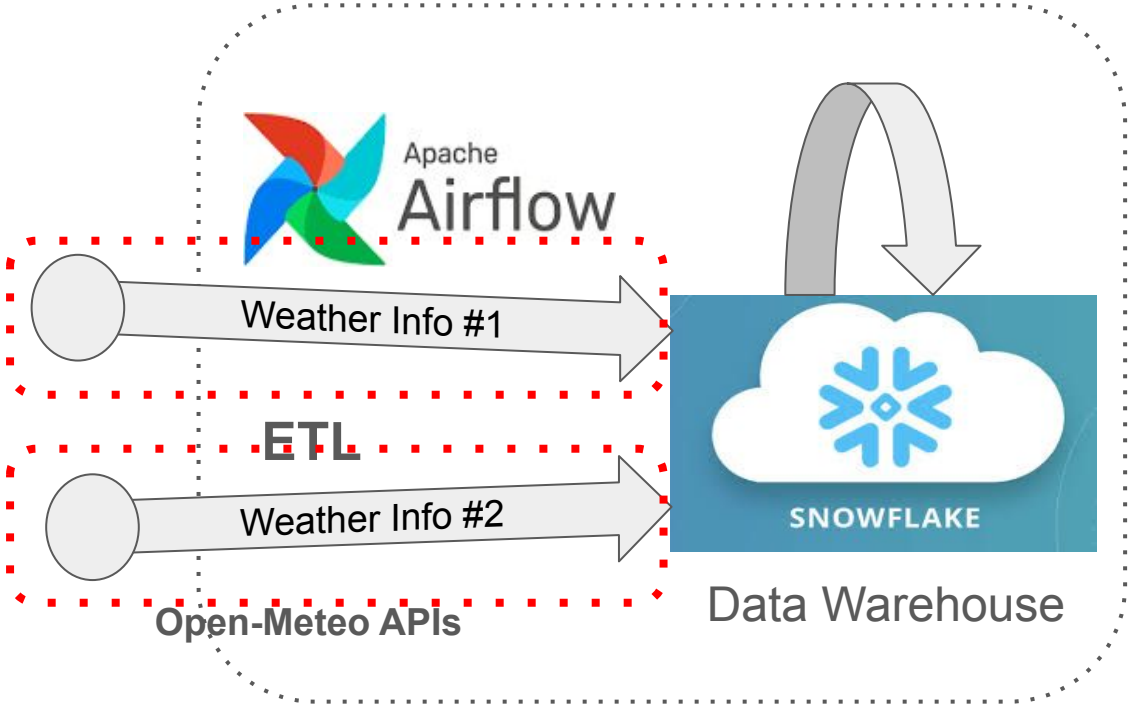
ETL v2 Practice

- Data Pipeline Example from Google Colab

More on Lab #1

Building a Weather Prediction Analytics using Snowflake & Airflow

Task to forecast the next 7 days of weather (**ELT**)



Group Project & Final Exam

Group Project Proposal

- 2/26: Group Project Team Selection
- 3/12: Group Project Proposal Due
- 5/7: Group Project Presentation

- Deliverables:
  - Project proposal
  - Project live presentation
  - Project code (GitHub repo)
  - Project final report

Midterm and Final Exam

- Midterm: 6 - 8:30 PM on Mar 26 (Thu)
  - Part A: 60 points, 50 minutes (30 multiple-choice and multiple-answer questions)
  - Part B: 85 points, 65 minutes (10 short- and long-answer questions; 3 of them will be on SQL)
- Final Exam: 5:30 - 7:30 PM on May 14 (Thu)
  - Part A: 60 points, 50 minutes (30 multiple-choice and multiple-answer questions)
  - Part B: 85 points, 65 minutes (10 short- and long-answer questions; 3 of them will be on SQL)

Resume Workshop (3/7)

Demo: Airflow Installation on Docker

Homework

Homework: Weather Temperature ETL in Python (15 pt)

- This homework should be done in Python using Google Colab. Use this link as a reference.
- Use Open Meteo API ("historical weather" API)

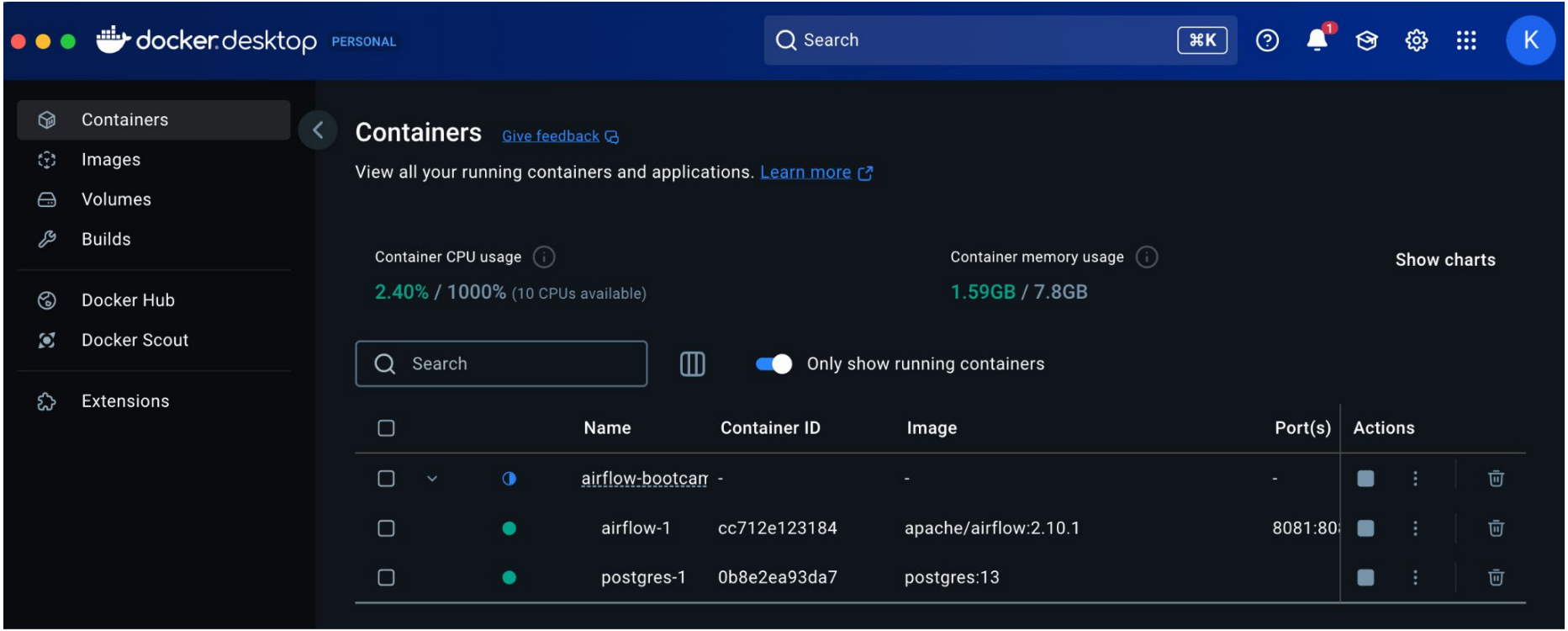
1. (+1) Pick up a city and get its lat/long
2. (+1) Secure your Snowflake credentials (don't expose them in the code)
3. (+2) Read the last 60 days of the weather info via the API (refer to the code snippet)
4. (+1) Create a table under "raw" schema if it doesn't exist to capture the info from the API
   a. latitude, longitude, date, temp_max, temp_min, precipitation, weather_code
   b. latitude, longitude and date should be **primary keys** (composite key)
5. (+1) Delete all records from the table
6. (+1) Populate the table with the records from step 2 using INSERT SQL
7. (+4) Steps 4 and 6 need to be done together
   a. Use try/except along with SQL transaction. (use the code here as reference)
8. (+1) Demonstrate your work ensures Idempotency by running your pipeline (from extract to load) twice in a row and checking the number of records (the number needs to remain the same)
9. (+2) Follow today's demo and capture Docker Desktop screen showing Airflow

(+1) Overall formatting (screenshots)

Step 9

https://github.com/keeyong/sjsu-data226-SP26/blob/main/docker-compose.yaml

Homework: Step 9 screenshot example



Next Week: Airflow!